Desarrollo full stack

Tareas

Entrega final del reto/proyecto

MVP

Isaac Isaias Betance

Git-Hub: https://github.com/YoIsaac/YoIsaac-task-manager-mvp-Proyecto-Final-Stack.git

Git-Hub Todo: https://github.com/YoIsaac/YoIsaac-task-manager-mvp-Proyecto-Final-Stack.git

Git-Hub Backend: https://github.com/YoIsaac/Backend-PF-DFS.git

Git-Hub Fronted: 🌐 YoIsaac/FRONTED-PF-DFS

ENTREGA FINAL – TASK MANAGER PRO

1. Descripción del Proyecto

Task Manager PRO es una aplicación Full Stack que desarrollé como evolución de mi MVP anterior. En la primera versión el backend funcionaba con datos en memoria, pero en esta fase final lo mejoré integrándolo con PostgreSQL usando Supabase.

-El sistema permite:

-Crear tareas

-Consultar tareas

-Actualizar tareas

-Eliminar tareas

-Autenticación de usuarios

-Manejo de roles

-Integración con una API externa

Este proyecto representa un avance importante en mi proceso de aprendizaje, ya que pasé de una aplicación básica a una arquitectura más completa y cercana a un entorno profesional.

2. Idea del Proyecto

La idea del proyecto es crear un administrador de tareas sencillo pero funcional, pensado para estudiantes o personas que necesitan organizar sus actividades diarias.

El problema que intenta resolver es la falta de organización y seguimiento de tareas en el día a día. A través de este sistema, el usuario puede gestionar sus pendientes de manera estructurada.

Tomé como base un gestor visto en clase, pero lo adapté y amplié con autenticación, base de datos real y despliegue en la nube.

3. Arquitectura del Sistema

En este proyecto utilicé la siguiente arquitectura:

Backend:

-Node.js

-Express

-PostgreSQL (Supabase)

-JWT para autenticación

-Middlewares personalizados

Frontend:

-Next.js

-Rutas dinámicas

-Consumo de API

-Manejo de token de sesión

Base de datos:

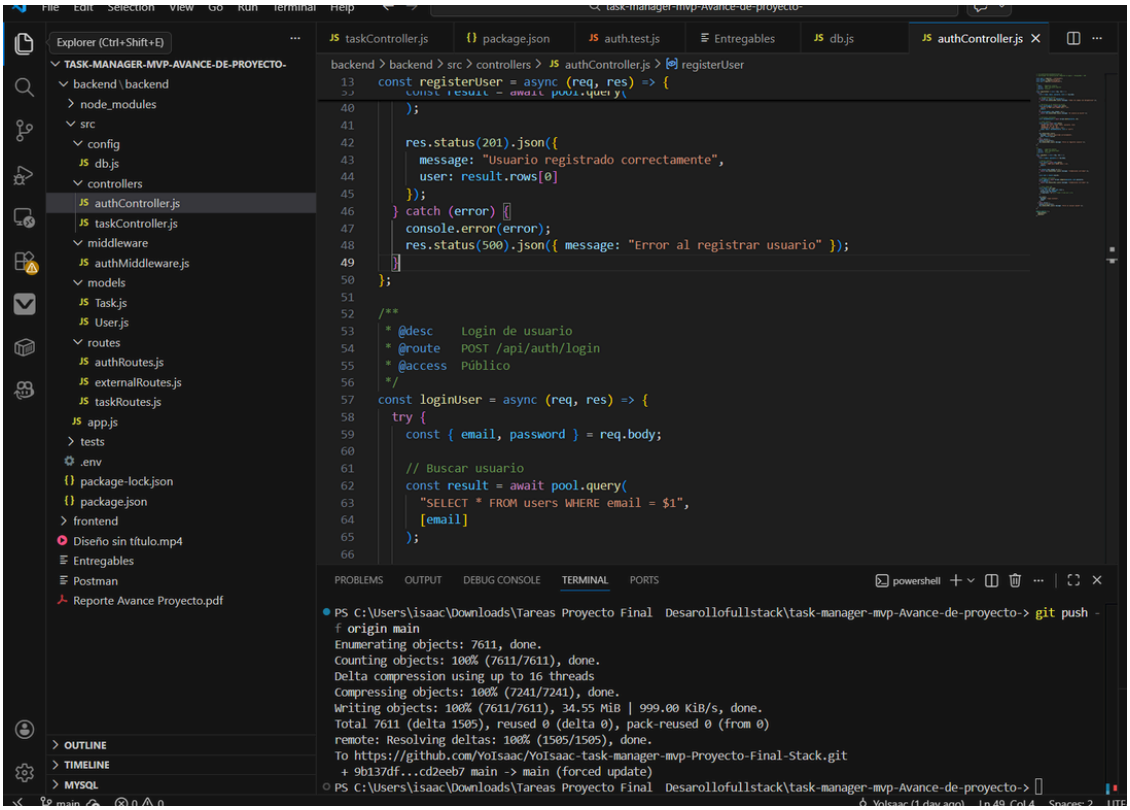
-Supabase (PostgreSQL en la nube)

Despliegue:

-Backend en Render (actualmente en proceso de ajustes)

-Frontend en Vercel (actualmente en proceso de ajustes)

En este momento el sistema funciona correctamente en entorno local y conectado a Supabase. El despliegue en producción está en proceso mientras termino de configurar correctamente las variables de entorno y detalles técnicos.



4. Backend – Mejoras Implementadas

Autenticación con JWT

Implementé un sistema de autenticación usando JSON Web Tokens.

El backend ahora incluye:

-Endpoint de login que genera un token

-Middleware que valida el token

-Protección de rutas

-Manejo de roles (admin y user)

Por ejemplo, la ruta /admin/users solo puede ser accedida por un usuario con rol administrador.

Middleware adicional

Agregué diferentes middlewares para mejorar la seguridad y organización del código:

-Validación de token

-Validación de rol

-Manejo centralizado de errores

-Respuestas HTTP correctas (400, 401, 403, 404)

Esto me ayudó a entender mejor cómo estructurar un backend de forma más profesional.

Paginación

Implementé paginación en el endpoint GET de tareas.

Ejemplo:
GET /tasks?page=1&limit=5

Esto permite consultar los registros por bloques y mejora el rendimiento cuando existen muchas tareas.

Para implementar esto investigué cómo funciona la paginación en Express con PostgreSQL y luego adapté la solución a mi estructura de proyecto.

5. Integración de API Externa

Integré una API pública de clima dentro del backend.

Se creó el endpoint:

-/external/weather

El backend realiza la consulta a la API externa y el frontend muestra la información obtenida.

También agregué manejo de errores en caso de que la API externa no responda correctamente.

6. Frontend – Next.js

En el frontend trabajé con Next.js utilizando el App Router.

Implementé:

-Ruta /tasks para listar tareas

-Ruta /tasks/[id] para detalle

-Página para crear tareas

-Página de login

-Manejo de token

-Consumo de API usando variable de entorno NEXT_PUBLIC_API_URL

El frontend se conecta al backend y muestra los datos en tiempo real.

A nivel visual aún estoy mejorando algunos detalles, pero la funcionalidad principal está implementada y operativa.

7. Pruebas y Debugging

Durante el desarrollo realicé:

-Pruebas manuales con Postman

-Validación de rutas protegidas

-Intentos de acceso sin token para verificar errores 401

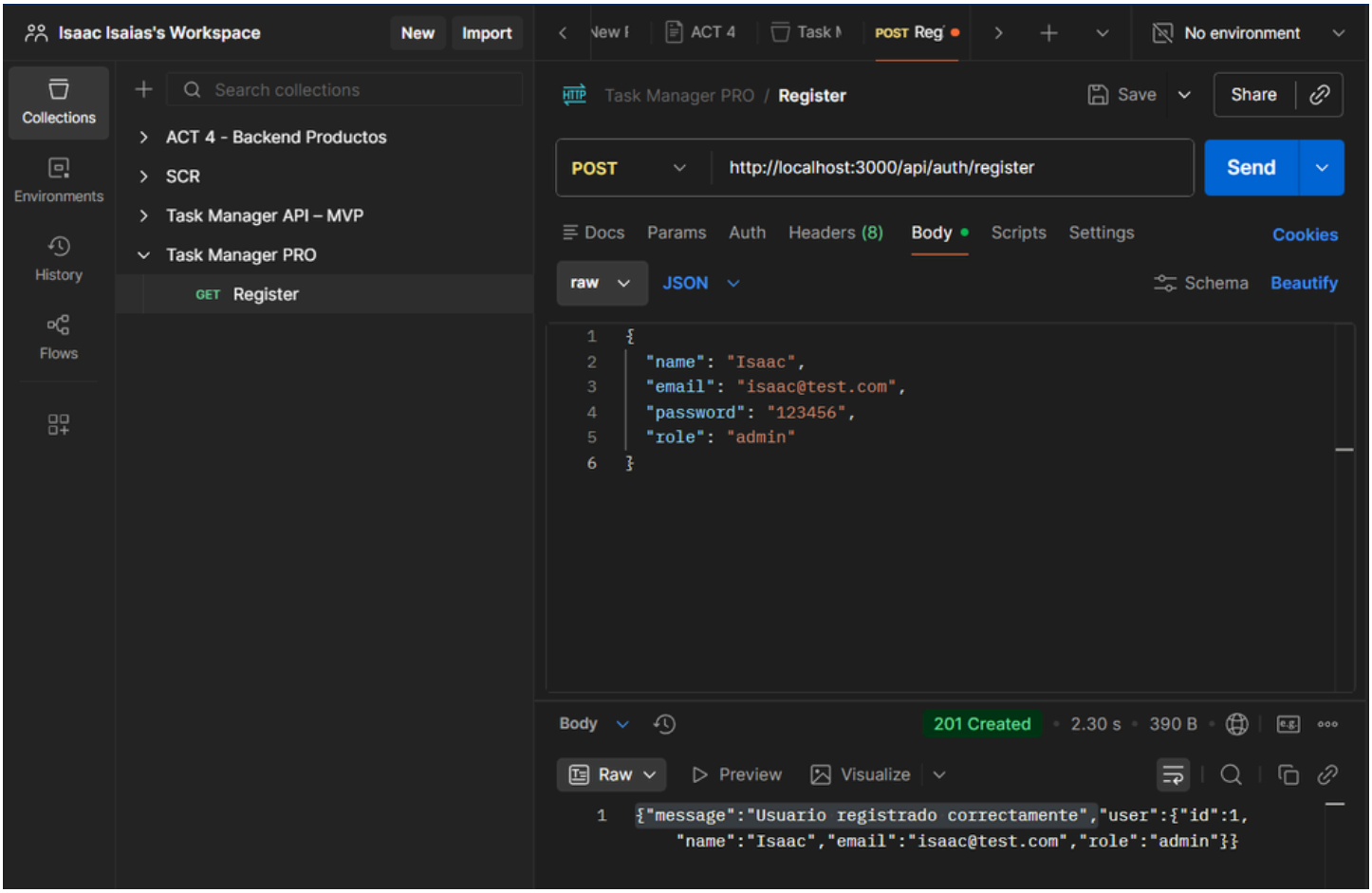
-Pruebas de roles

También enfrenté algunos problemas técnicos durante el despliegue, especialmente relacionados con configuración y variables de entorno. A través del debugging y revisión de logs pude identificar los errores y avanzar en su solución.

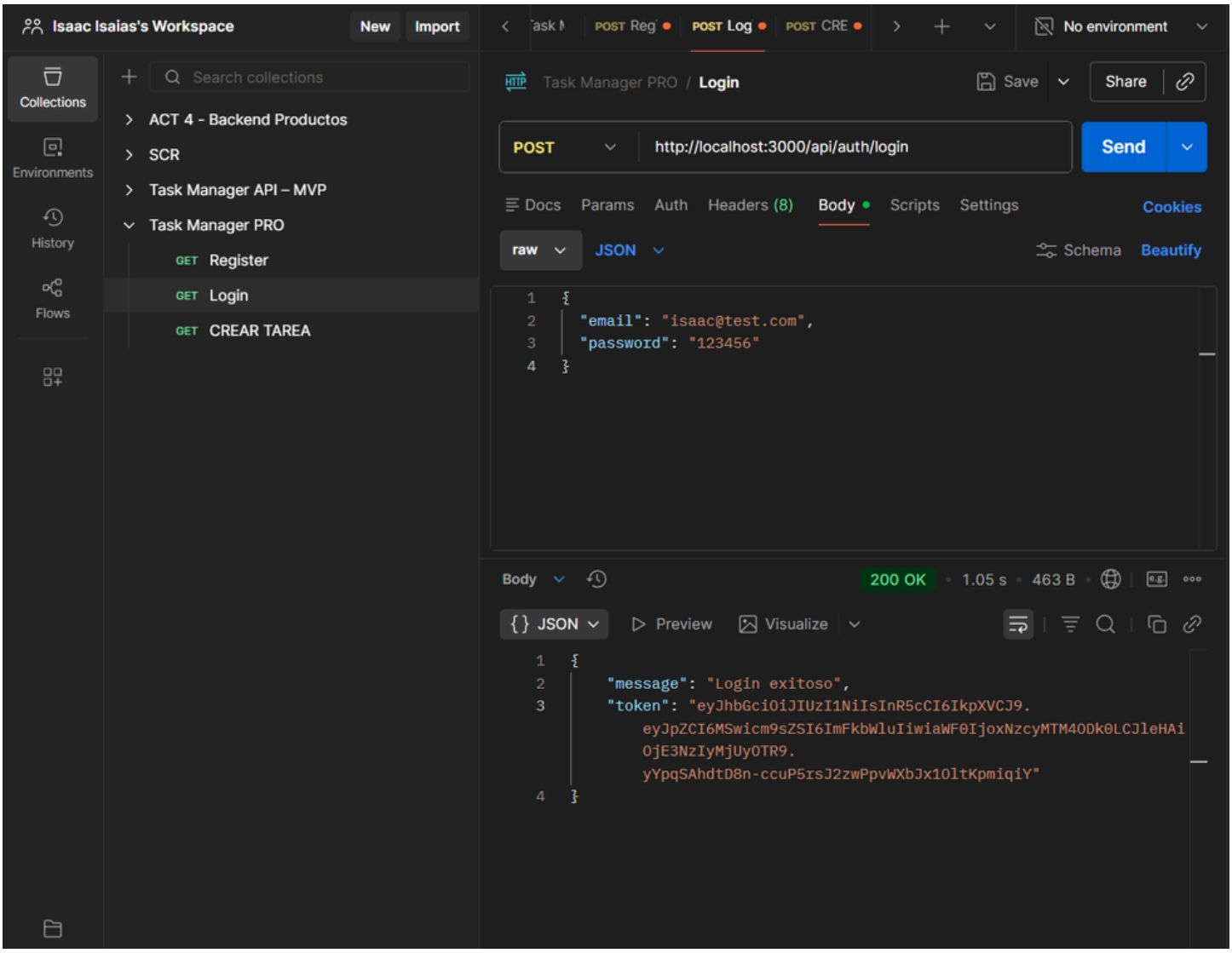
Actualmente el proyecto funciona correctamente en entorno local. El despliegue final en Render y Vercel está en proceso mientras termino de ajustar configuraciones.

PON AQUÍ CAPTURA DE TERMINAL CORRIENDO SERVIDOR
PON AQUÍ CAPTURA DE ERROR 401 SIN TOKEN

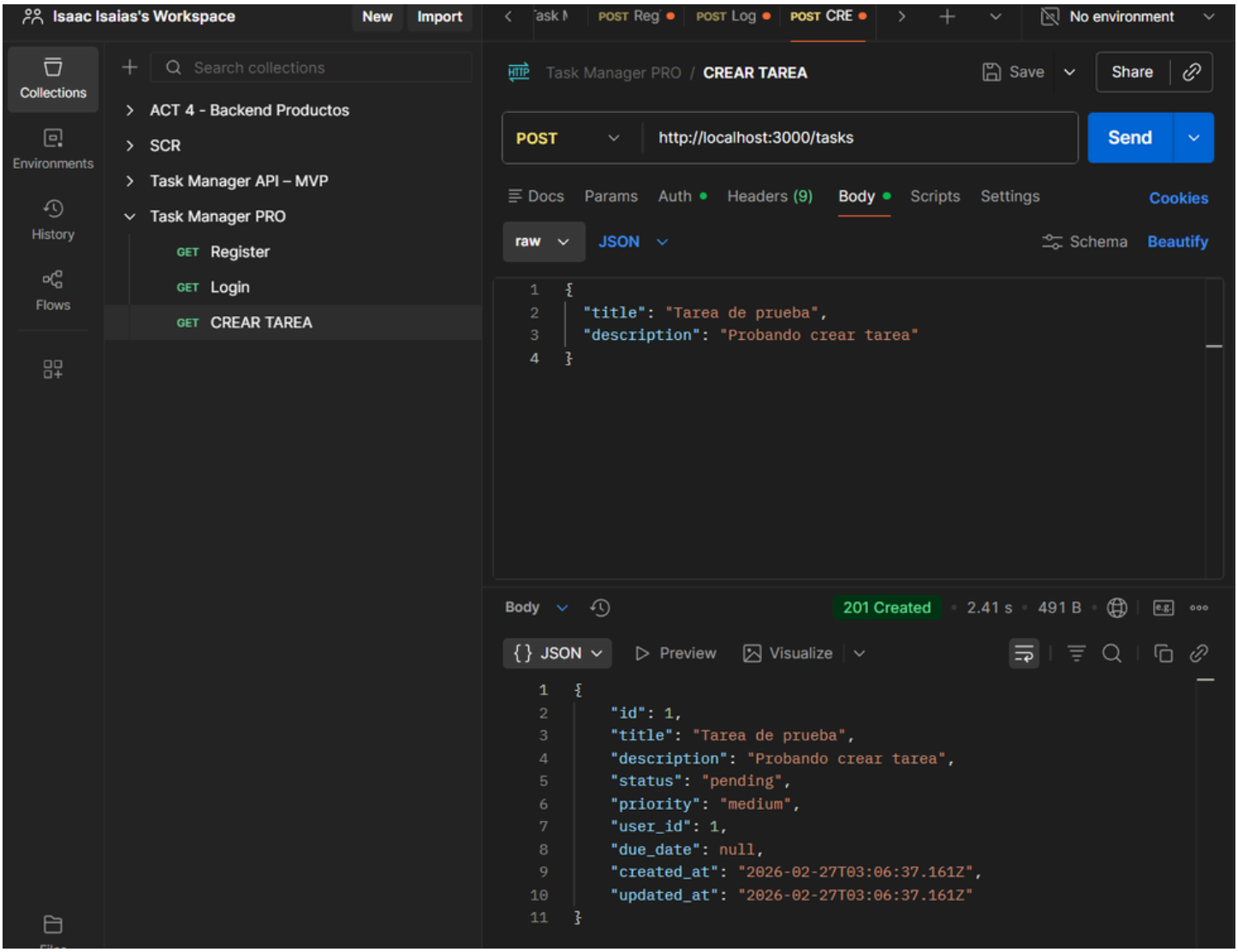
Register: Se probó el registro de un nuevo usuario enviando nombre, email y contraseña y verificando que se guardara correctamente en la base de datos.



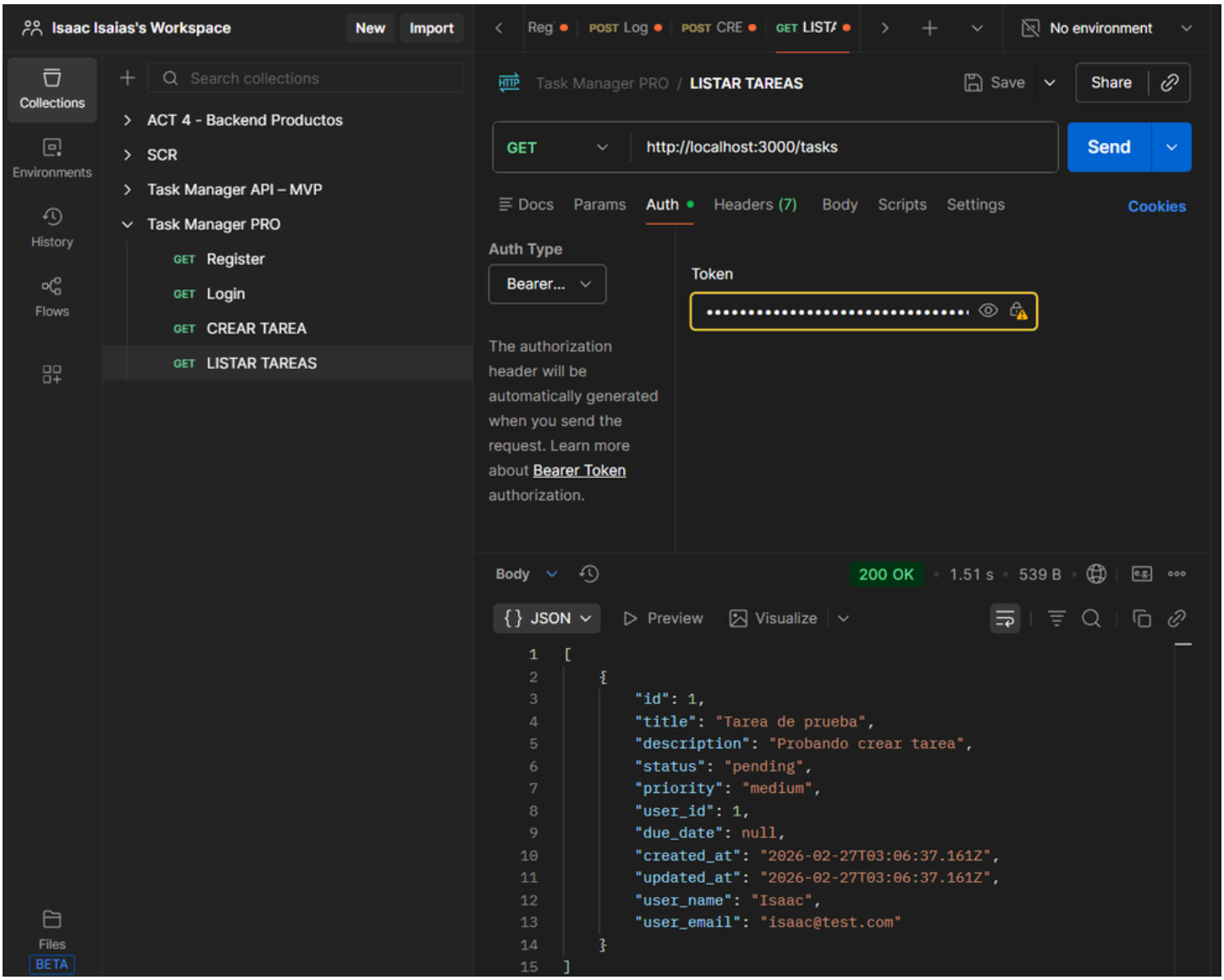
Login: Se validó el inicio de sesión con credenciales correctas y se comprobó que el servidor generara un token JWT válido.



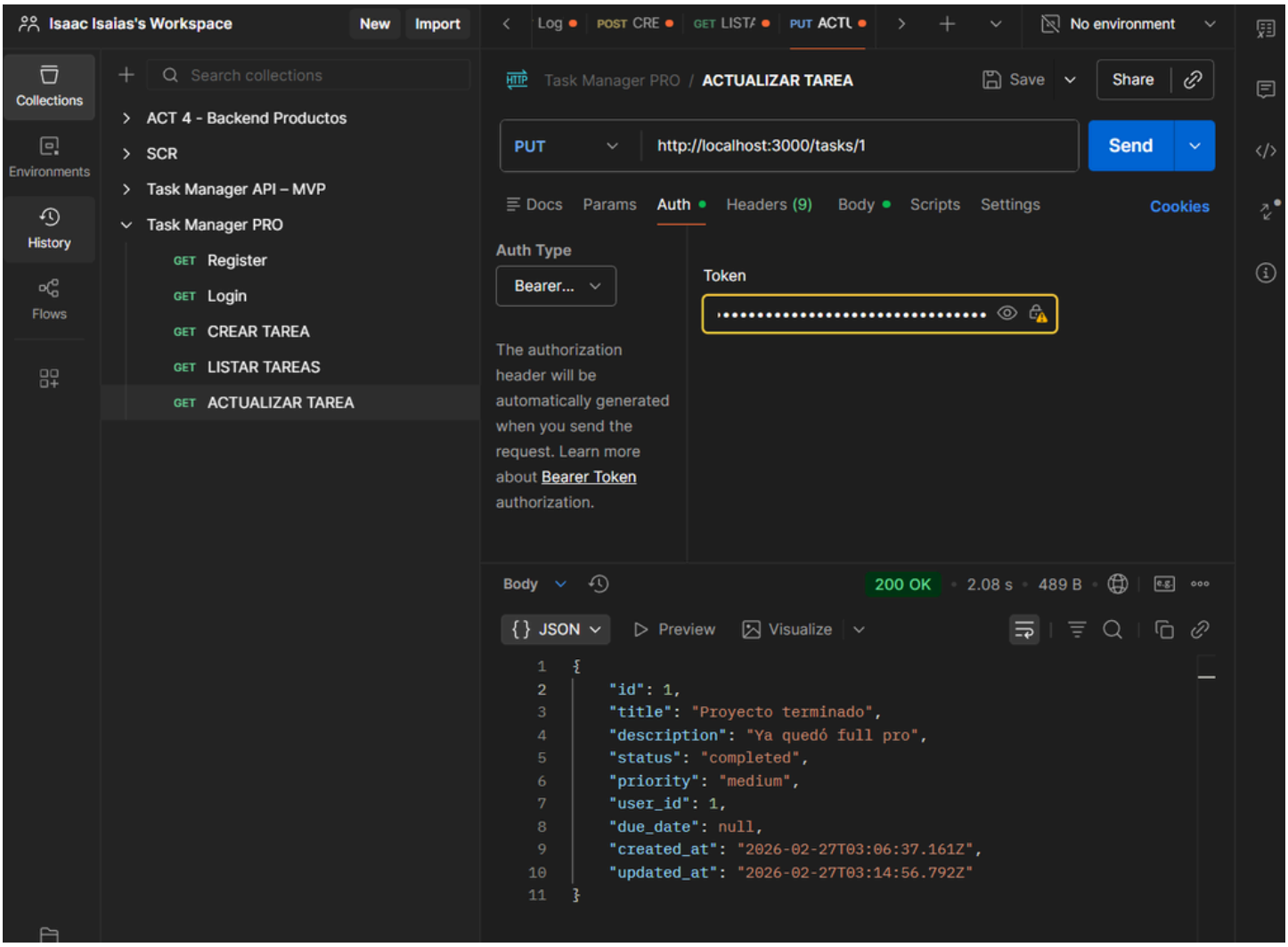
Crear Tarea: Se envió una solicitud POST con token en el header Authorization para crear una nueva tarea asociada al usuario autenticado.



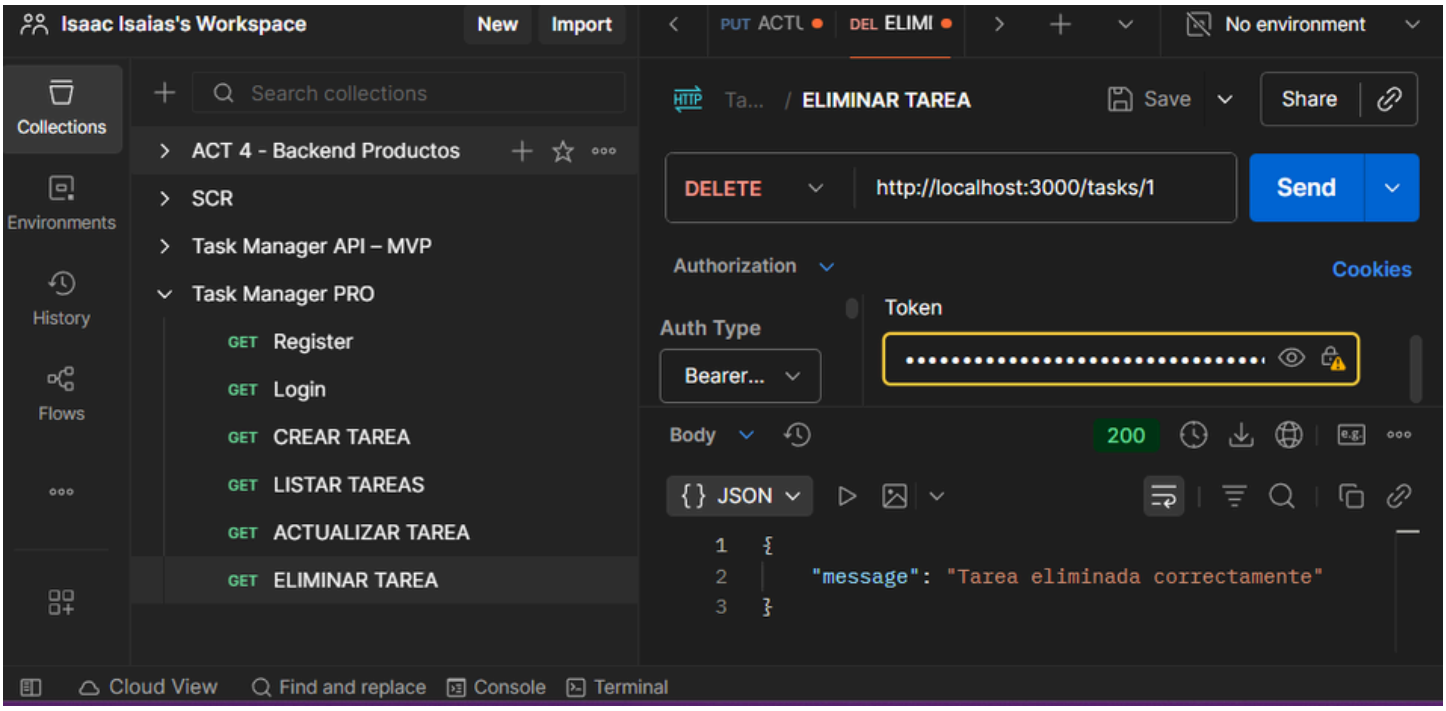
Listar Tareas: Se realizó una petición GET con token para obtener todas las tareas registradas del usuario.



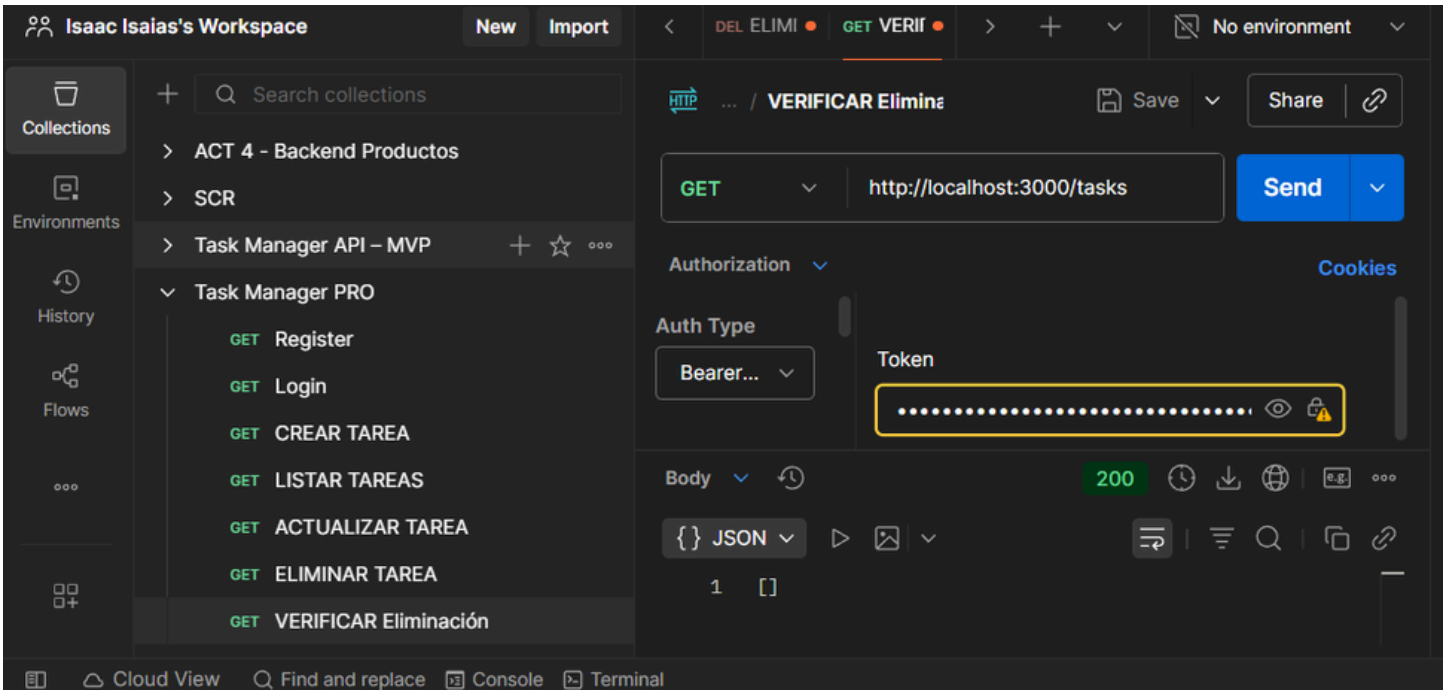
Actualizar Tarea: Se probó el endpoint PUT enviando nuevos datos para una tarea específica y verificando que los cambios se reflejaran en la base de datos.



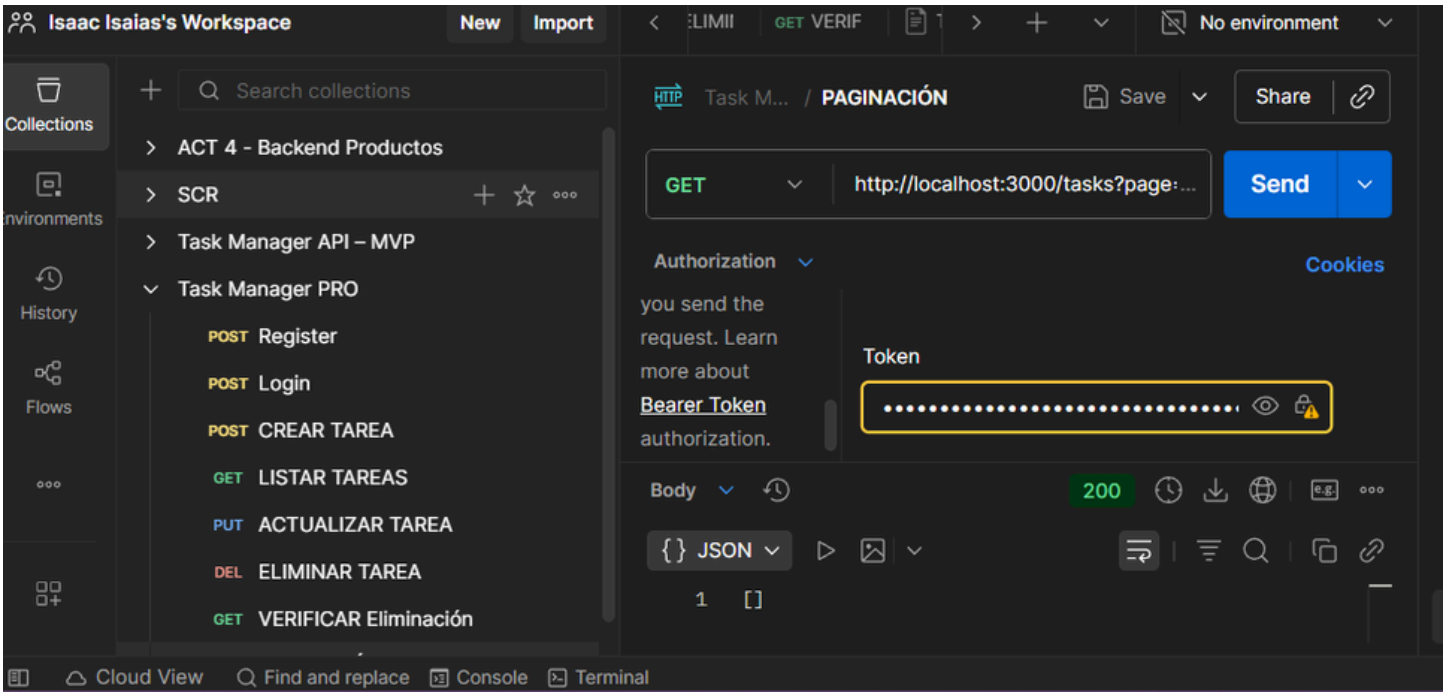
Eliminar Tarea: Se ejecutó una solicitud DELETE sobre una tarea existente y se confirmó que el servidor respondiera correctamente.



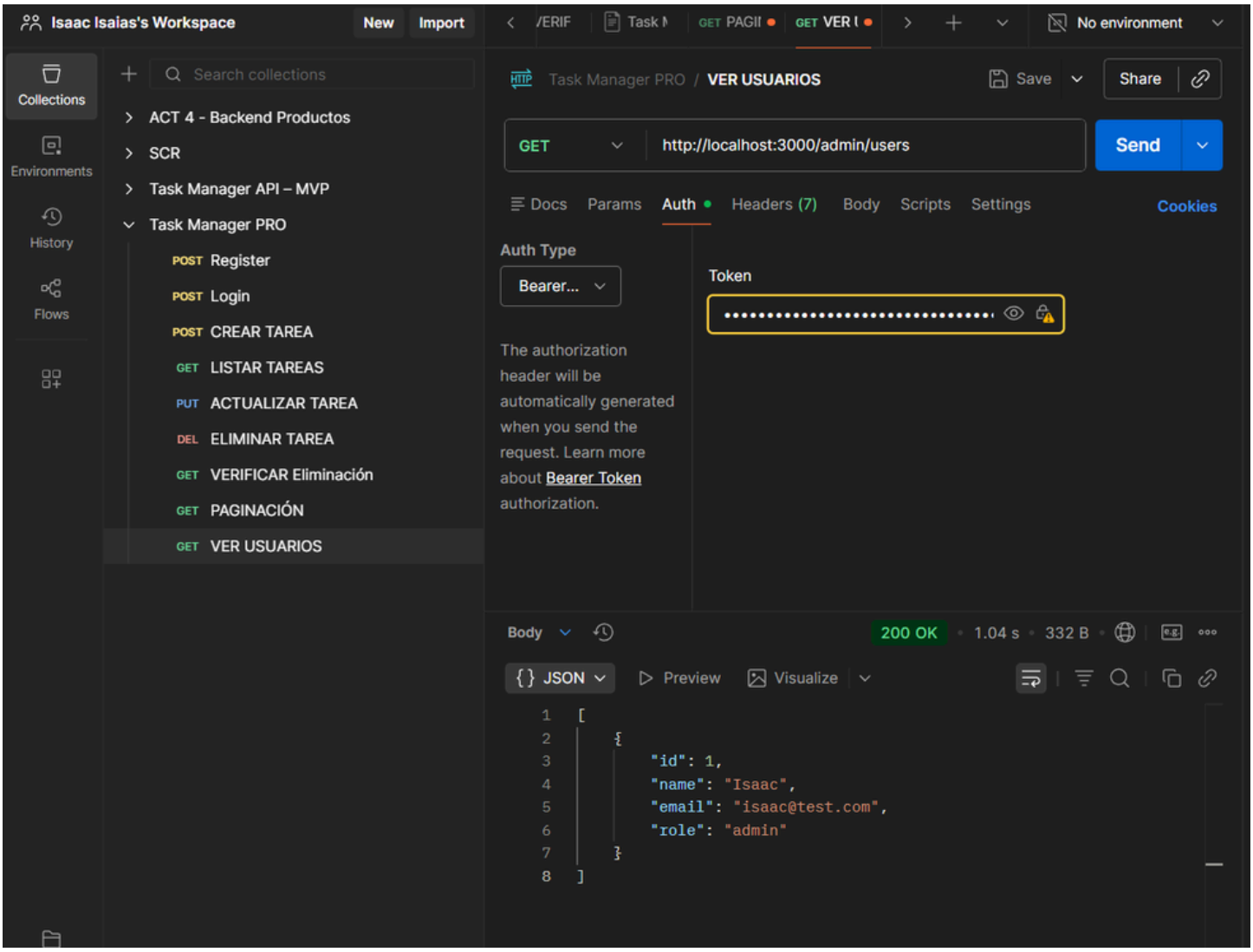
Verificar Eliminación: Después de eliminar una tarea, se volvió a consultar la lista para comprobar que ya no apareciera en los resultados.



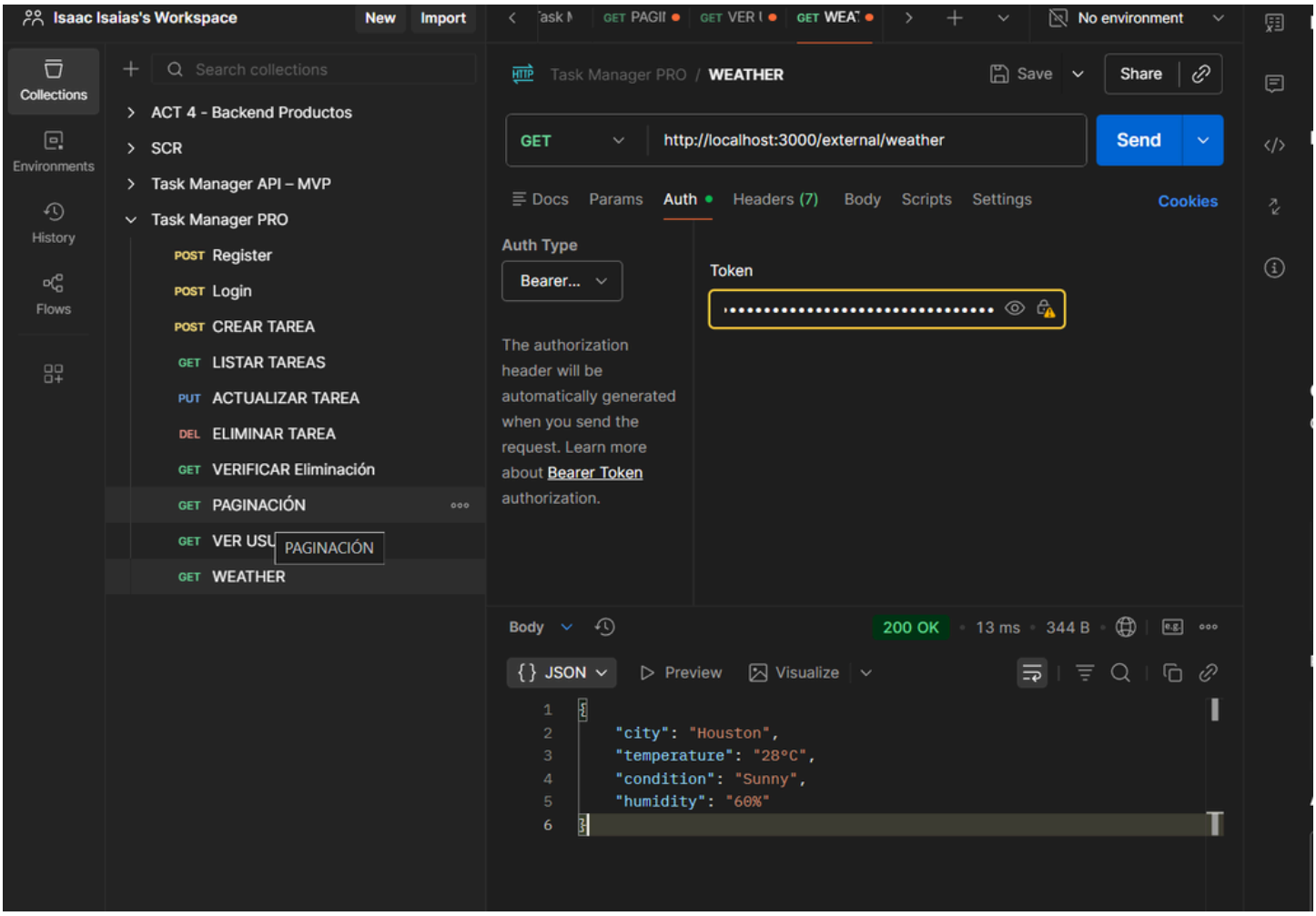
Paginación: Se utilizó el endpoint con query params ?page=1&limit=5 para validar que los resultados se dividieran correctamente por páginas.



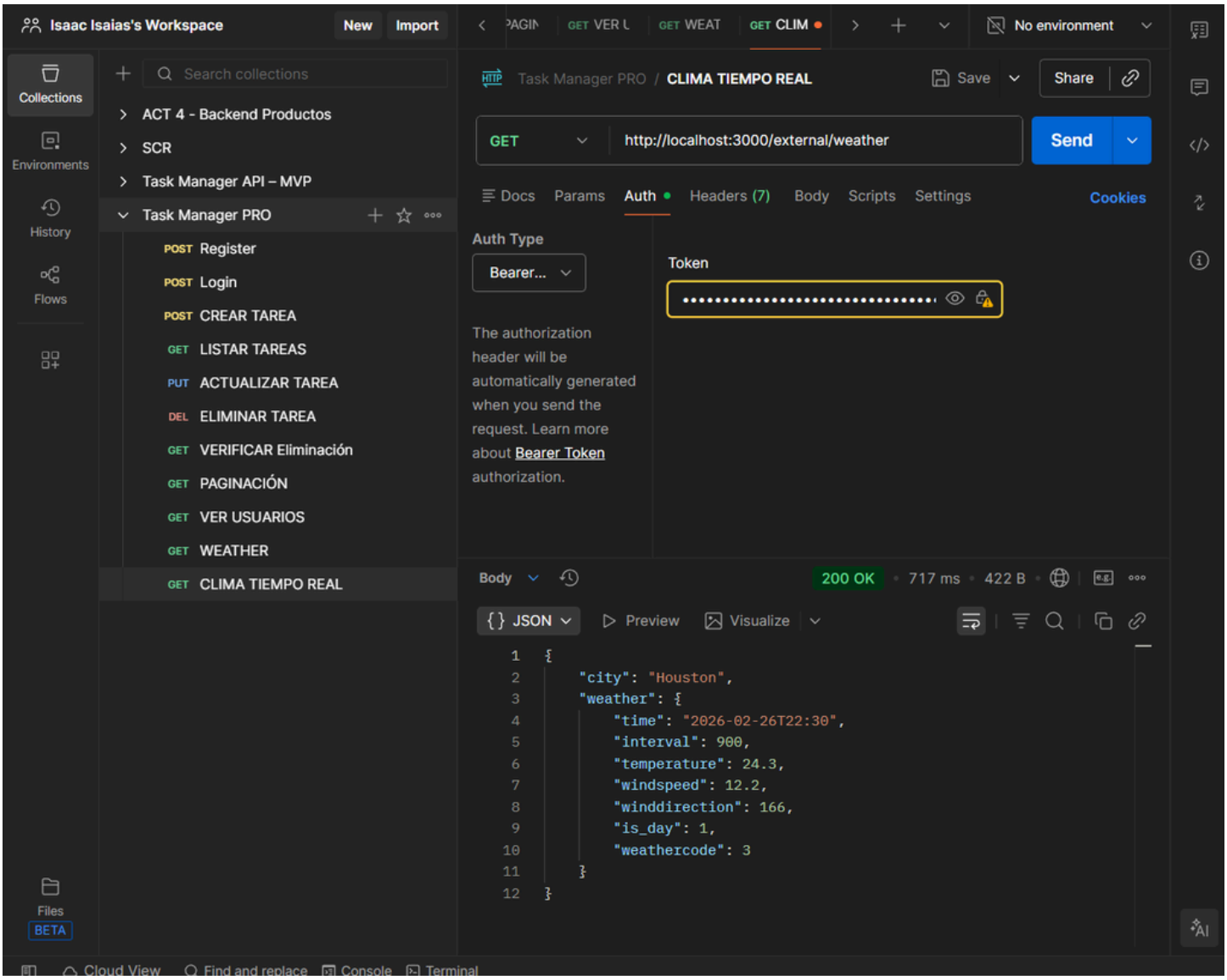
Ver Usuarios: Se probó la ruta protegida de administración para listar usuarios, confirmando que solo el rol admin pudiera acceder.



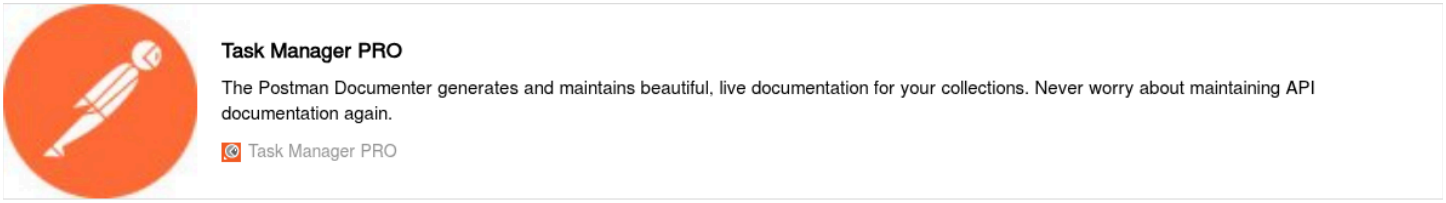
WHEATER: Se probó el endpoint que consume la API externa para validar que el backend recibiera y procesara correctamente la información del clima.



Clima Tiempo Real: Se verificó desde el frontend que los datos obtenidos del endpoint /external/weather se mostraran correctamente en pantalla.



Postman: https://documenter.getpostman.com/view/51943433/2sBXcHgyJ1



8. Despliegue en Producción (En Proceso)

Estoy trabajando en:

-Configuración del backend en Render

-Configuración del frontend en Vercel

-Variables de entorno

-Conexión correcta con Supabase en producción

Todavía estoy en proceso de ajuste, pero el objetivo es que quede completamente funcional en la nube.

Este proceso me ha permitido entender mejor cómo funcionan los entornos de producción y la diferencia entre local y producción.

9. Credenciales de prueba

Admin:
email: admin@test.com
password: 123456

Usuario:
email: user@test.com
password: 123456

10. Conclusión

Este proyecto representa un avance importante en mi aprendizaje como desarrollador Full Stack.

Logré:

-Migrar de datos en memoria a PostgreSQL

-Implementar autenticación con JWT

-Proteger rutas por rol

-Integrar una API externa

-Construir un frontend funcional con Next.js

-Trabajar en despliegue en la nube

Aunque el despliegue final aún está en proceso de ajustes, el proyecto cumple con los requerimientos técnicos principales y demuestra una mejora clara respecto al MVP inicial.

Agregare las evidencias de Render y lo que me falta de fronted a git-hub ya estan pero tengo problemas para hacerlos funcionar por ahora.